ULTIMATE CYBERSECURITY MASTER BLUEPRINT 2026-2027

Volume 6 — Lab, Projects, Assessments and Career

Home lab, 15 projects, scorecards, certification plan, GitHub and international preparation

Edition: Volume 6
Resource verification date: 2026-10-27
Plan window: 2026-10-27 to 2027-10-26 (shift dates if you start later)
System: 365 days, 12 academic months, 52 weeks + 1 graduation day
Profile: BS CS student in Pakistan; free/high-value resources; safe authorized labs

Academic-style self-study manual. No employment, salary, visa, certification or top-1% guarantee.

Table of contents

Volume 6 — Lab, projects, assessments, certifications, portfolio and career 3
1. Affordable home lab 3
2. Project system 4
3. Assessments and mastery gates 5
4. Certification planning (verified 2026-10-27) 6
5. GitHub and professional portfolio 8
6. International career preparation 8
Specialization comparison (Month 8-10 decision aid) 9

Project catalogue — 15 required deliverables 10
PRJ-01 — Personal cybersecurity learning lab 10
PRJ-02 — Security control baseline and risk register 10
PRJ-03 — Network diagram and troubleshooting report 11
PRJ-04 — Wireshark traffic-analysis report 12
PRJ-05 — Linux system-audit script 12
PRJ-06 — Windows event-log investigation 13
PRJ-07 — Python security-log parser and defensive automation tool 14
PRJ-08 — Secure web application / API 14
PRJ-09 — Local web application security assessment with remediation 15
PRJ-10 — Authorized vulnerability assessment report for an isolated lab 16
PRJ-11 — SIEM alert investigation and incident report 16
PRJ-12 — Detection rule with test data 17
PRJ-13 — Cloud configuration security review 18
PRJ-14 — Docker image and container security review 18
PRJ-15 — Final integrated capstone 19
Printable trackers 20

Volume 6 — Lab, projects, assessments, certifications, portfolio and career

1. Affordable home lab

Minimum hardware

- CPU: 64-bit dual-core with virtualization (Intel VT-x/AMD-V); 4 threads recommended.
- RAM: **8 GB minimum** (host + one 2 GB Linux VM + browser labs). 16 GB strongly recommended; 32 GB for Windows Server/AD/Kubernetes.
- Storage: **80 GB free minimum**, 256 GB SSD recommended for snapshots.
- OS: Windows, Linux or macOS host; VirtualBox/KVM/Hyper-V.
- Network: intermittent internet for downloads; keep labs isolated offline.

Recommended hardware

- 4-8 core CPU, 16-32 GB RAM, 512 GB SSD, external backup drive.
- One Ubuntu LTS VM (2-4 GB RAM), one Windows evaluation VM (4 GB RAM), Docker Engine on Linux, optional kind/minikube cluster.
- Browser-based labs when RAM is low: PortSwigger, TryHackMe free rooms, pwn.college workspace, CyberDefenders datasets.

Official installation sources

- VirtualBox manual: https://www.virtualbox.org/manual/
- Ubuntu help: https://help.ubuntu.com/
- Docker Engine docs: https://docs.docker.com/
- Kubernetes docs / kind / minikube: https://kubernetes.io/docs/home/, https://kind.sigs.k8s.io/, https://minikube.sigs.k8s.io/docs/
- Windows evaluation / developer VMs: use only Microsoft's official pages https://www.microsoft.com/en-us/evalcenter/ and https://developer.microsoft.com/en-us/windows/downloads/virtual-machines/
- Wireshark: https://www.wireshark.org/docs/wsug_html_chunked/
- Packet Tracer: https://www.netacad.com/courses/packet-tracer
- OWASP Juice Shop: https://github.com/juice-shop/juice-shop
- DVWA: https://github.com/digininja/DVWA
- Kubernetes Goat: https://madhuakula.com/kubernetes-goat/

Network isolation

Use NAT for updates, host-only/internal adapters for vulnerable services, and never bridge intentionally vulnerable VMs to a university/hostel/public LAN. Bind local web apps to `127.0.0.1` or an internal adapter only. Do not expose lab ports on your router. Keep host firewall/AV enabled; do not disable essential security controls as routine practice. Take a clean snapshot before each risky lab and write down the restore point. Use synthetic data, not personal files.

VM vs container vs browser lab

Need	Best tool	Why
OS internals, Active Directory, Windows event logs, snapshots, malware-like artifacts	Full VM	Kernel/identity state, snapshot/rollback, stronger isolation
Stateless vulnerable web app, scanner, parser	Container	Fast, reproducible, lower RAM
Low-end laptop / public Wi-Fi / no admin rights	Browser lab	No install; platform-provided isolation
Multi-node Kubernetes	kind/minikube on 16 GB+ host	Local control plane, no cloud bill

Need	Best tool	Why
Real cloud API behaviour	Free-tier cloud account with budgets	Only when local emulation cannot show the service

Avoid real malware on the host. Malware-traffic datasets are PCAPs/artifacts, not permission to execute suspicious binaries. For any RE/malware lab in Year 2+, use a disposable VM with no shared clipboard, no bridged adapter and no personal credentials.

Cloud cost safety

1. Read current Free Tier terms (https://aws.amazon.com/free/) before account creation.
2. Enable MFA on root/owner accounts; never commit access keys.
3. Create billing alerts/budgets before deploying resources.
4. Tag every resource with project and expiry date.
5. Prefer local/synthetic cloud reviews. If you deploy, capture evidence then tear down the same day.
6. Check the billing page and delete unused volumes, snapshots, IPs and clusters; these often cost money even when VMs are stopped.
7. Never paste cloud credentials into GitHub, screenshots or PDFs.

Common setup problems

Symptom	Likely cause	Fix
VM will not start	Virtualization disabled in BIOS/UEFI or Hyper-V conflict	Enable VT-x/AMD-V; check host hypervisor settings; use VirtualBox manual network chapter
No internet in VM	Wrong adapter/DNS/DHCP	NAT for updates; verify `ip addr`, `/etc/resolv.conf`, DHCP lease; do not disable firewall globally
SSH timeout	Guest IP changed, service not running, host-only route	Check `ip addr`, `systemctl status ssh`, `ss -ltn`, and VM adapter
Port conflict on host	Another service bound to 8080/3000	Use `ss -ltnp` or `netstat -ano`, choose another loopback port
Docker permission denied	User not in docker group or daemon not running	Use official docs; avoid `curl | sh` scripts from random sources; consider rootless Docker
Windows events missing	Audit policy/Sysmon config not applied	Verify Event Viewer, audit policy and Sysmon service; reapply in a clean snapshot
Kubernetes cluster high RAM	Too many nodes/control-plane components	Use single-node kind/minikube, 2-4 GB RAM, stop cluster after lab
Packet Tracer file missing	Registration/download issue	Use official NetAcad course; backup with network diagrams and Wireshark local captures
Cloud surprise bill	Free-tier expiry, orphaned storage/IPs	Budget alerts, tags, daily teardown, billing dashboard check

2. Project system

Every month has at least one deliverable. All projects use synthetic data, isolated labs and systems you own or are explicitly authorized to test. Every project section includes problem, objective, prerequisites, architecture, tools, milestones, expected results, validation, common mistakes, troubleshooting, evidence, report template, GitHub structure, README, security boundaries, rubric and extension challenges.

Shared GitHub structure

```
project-name/
  README.md
  LICENSE
```

```
CHANGELOG.md
.gitignore
docs/
  problem.md
  architecture.md
  threat-model.md
  report.md
  limitations.md
src/ or manifests/
tests/
data/synthetic/
evidence/sanitized/
scripts/setup.sh
```

README template

1. One-sentence problem statement; 2. Scope/authorization statement; 3. Architecture diagram; 4. exact setup commands; 5. usage examples; 6. tests/validation; 7. findings/results; 8. limitations and lessons learned; 9. sanitization notes; 10. licence/acknowledgements.

Report template

Executive summary → scope and rules → architecture/data → methodology → findings with evidence → risk/priority → remediation → retest → residual risk → timeline → appendices. Clearly label facts, interpretations and assumptions. Never publish secrets, private personal data, sensitive logs or unauthorized exploit material.

Common project rubric (100 points)

Conceptual understanding 20; practical execution 25; independent problem-solving 15; troubleshooting 15; documentation 15; safety/ethics 10. Passing = 70, with safety ≥9 and practical ≥18. Project-specific validation tests in each card override generic claims.

3. Assessments and mastery gates

Monthly practicals

Month	Timed practical	Critical prerequisite gate
M1	Configure clean VM, users, local service, SQL query, Git evidence	CLI, permissions, VM isolation
M2	20 subnetting questions, unknown PCAP, broken topology, scope judgement	OSI/TCP/IP, CIDR, Wireshark, authorized Nmap
M3	Linux/Windows hardening, event-log timeline, Bash/PowerShell script	Users/processes/logs, identity boundaries
M4	Unseen log format parser, tests, normalized schema, short report	Python, regex, files, Git, testing
M5	Risk register, crypto primitive selection, IAM design, IR decision	CIA/AAA, crypto purpose, risk model
M6	Find/fix two local web bugs, API authorization design, remediation report	HTTP, sessions, server-side controls
M7	Scoped local assessment, evidence, validation, report, retest	Written scope, no unauthorized targets
M8	Synthetic alert investigation, KQL/Sigma rule, incident report	Log normalization, ATT&CK, confidence
M9	Cloud/K8s/Docker/CI/IaC misconfiguration review	IAM, shared responsibility, isolation
M10	Unfamiliar specialization problem + technical defence	Primary docs, independent debugging
M11	Capstone implementation, failure injection, investigation, report	Full stack, threat model, evidence
M12	Three final practicals + capstone defence	All gates

Six-step failure repair

1. **Identify the gap:** score each rubric category and write the exact failed claim/command.
2. **Revisit primary lesson:** return to today's exact section; do not rewatch the whole course.
3. **Use alternate explanation:** use the supplementary resource or a different medium (diagram, RFC, man page).
4. **Repeat exercise:** same objective, new input; record prediction before result.
5. **New exercise:** solve a modified problem with no worked solution and write a short postmortem.
6. **Retest:** new 10-question or 30-minute practical; require ≥80% and no safety error before proceeding.

If two consecutive attempts fail, reduce scope to a prerequisite module for three days and notify your mentor/peer. Do not hide failed attempts; error logs are evidence of learning.

4. Certification planning (verified 2026-10-27)

Prices are official list prices where displayed; taxes, currency, region and promotions can change. Always re-open the official page before paying. No proprietary exam questions are reproduced.

Certification	Current official title/ version	Prerequisites	Exam cost shown	Training cost	Renewal/ maintenance	Consider	What it does not prove	Free alternative
ISC2 CC	Certified in Cybersecurity; outline effective 2026-09-01	No work experience; application/ endorsement process	US$199 in official pricing table for listed regions incl. Asia Pacific; local tax/ currency at Pearson	Official self-paced/ live training is paid; free flashcards/ quiz/ outline exist	Annual maintenance/ CPE obligations apply; verify current CPE handbook	Month 12 if a voucher or an employer/ university requests it	Hands-on SOC/ AppSec ability; real incident judgement	CS50 Cyber, Professor Messer, ISC2 free flashcards, this roadmap
CompTIA Security+	V7 SY0-701 current; V8 SY0-801 expected 2026-11-17; V7 English retirement listed 2027-06-11	Recommended Network+ plus ~2 years admin/ security experience, but no formal prerequisite	V7 voucher shown US$404 on CompTIA price page/ blog; verify current store and V8 price	CertMaster Learn/ Labs/ Practice paid; free Professor Messer videos	3-year cycle, 50 CEUs or higher cert/ CertMaster CE; official CE fee listed US$150 over 3 years	Only if a job/degree filter explicitly asks; prefer V8 after objectives settle	Deep technical execution; practical exams are limited	Professor Messer SY0-701, NIST/ISC2 free resources, local labs
Cisco CCST Cybersecurity	100-160 CCST Cybersecurity	None	US$125, 50 minutes	Free Cisco Junior Cybersecurity Analyst career path; paid instructor options exist	Cisco states 5-year validity for certifications earned from 2025-07-15; verify current rules	Month 5-12 if local employer values Cisco credentials	Broad hands-on engineering	Cisco free career path + this roadmap's networking/ security labs
Microsoft SC-900	Security, Compliance and Identity Fundamentals	Beginner; Azure/M365 familiarity helpful	Region-based; US price commonly listed as US$99 but verify at checkout	Microsoft Learn learning paths free	Fundamentals certification currently does not expire per Microsoft policy; verify	Optional after M5/ M9 if targeting Microsoft ecosystem	General security skill; product-specific knowledge	Four free SC-900 Learn paths, free practice assessment

Certification	Current official title/ version	Prerequisites	Exam cost shown	Training cost	Renewal/ maintenance	Consider	What it does not prove	Free alternative
Microsoft SC-200	Security Operations Analyst Associate	SOC/KQL/ Sentinel/ Defender experience recommended	Region-based; US associate price typically US$165 but verify official schedule page	Microsoft Learn free; instructor/ practice options paid	Annual free renewal assessment on Microsoft Learn	Year 2 if you choose Microsoft-aligned detection engineering	Vendor-independent detection engineering	KQL docs, Sigma, CyberDefenders, BTLO, local SIEM-like labs
AWS Security Specialty	SCS-C03 AWS Certified Security - Specialty	Official recommendation: ~5 years IT security and 2+ years AWS security	US$300 on official exam page	AWS Skill Builder free/paid; labs may cost	3-year validity; official recertification now lists exam or Skill Builder maintenance path—verify in your account	Year 2+ after real AWS work, not Month 12	General cloud security or hands-on skill	AWS Security Pillar, free Skill Builder digital courses, local IaC reviews
BTL1	Blue Team Level 1 (Centri, formerly Security Blue Team)	0-2 years recommended	£399 GBP training/exam bundle, one free resit included (official page)	Included in bundle	Lifetime certification per official page	Year 2 if employer funds it and you already completed free blue-team projects	Production SOC judgement	CyberDefenders, BTLO free labs, Autopsy, Sigma/ KQL, PRJ-11/12
eJPT	INE Junior Penetration Tester	Basic networking/ Linux/ Windows; no formal prerequisite	Official page says INE subscription + voucher required; checkout lists voucher/ bundle prices; verify current total	INE subscription/bundle paid	Valid three years per official page	Optional after M7 if you can afford it and have local reports	Professional scoping / reporting under real clients	NIST SP 800-115, local VMs, HTB Academy free modules, PRJ-10
OSCP + / PEN-200	OffSec Certified Professional / OSCP+	Strong TCP/ IP, Linux/ Windows, scripting recommended	Official page: US$1,699 standalone exam; US$1,749 course+exam bundle	Included in PEN-200 bundles; Learn One US$2,749/ year	OSCP lifetime; OSCP+ expires after 3 years unless maintained	Year 2-3 only with consistent independent pentest practice	Vulnerability research or elite status	pwn.college, HTB, local AD labs, report writing
CompTIA CySA+ / PenTest+ / GIAC / CISSP	Various intermediate/advanced credentials	Depends on credential; CISSP requires experience	Check each official page; not recommended this year	Often high	Varies	Not Year 1 priorities	Job-ready skill by themselves	Free blue/ red projects and primary documentation

Readiness checklist: current official objectives downloaded; ≥80% on two free/self-generated practice exams; portfolio evidence mapped to each domain; budget includes retake/renewal; reason for exam is written; practical projects already demonstrate skill. Practice strategy: closed-book timed attempt → mark confidence on each answer → error taxonomy (concept, wording, arithmetic, fatigue) → targeted revision → new self-generated questions → spaced retest after 3 and 7 days. Never use brain dumps.

5. GitHub and professional portfolio

A strong portfolio is not a folder of screenshots. It shows a problem, a decision, evidence, a failed attempt, a fix, a test and a limitation. Use meaningful commits, issues, tags and release notes. Add architecture diagrams as SVG and sanitized screenshots; redact IPs, MACs, usernames, hostnames, tokens, private paths and personal data. Include an explicit scope statement on security work. Never publish credentials, secrets, private logs or unauthorized exploit material.

Superficial project: copied Dockerfile, no README, one happy-path screenshot, no tests, no limitations. **Original project:** states a problem, threat model, 2+ design alternatives, implementation rationale, reproducible setup, negative tests, false-positive analysis, before/after evidence, lessons learned and a future-work section.

Recommended public index: one `portfolio` README linking 5-8 best projects (not all 15 if quality is uneven), a CV, a lab safety statement, and contact information. Keep private evidence encrypted.

6. International career preparation

Broad market patterns, not employer guarantees

Entry-level security work in US/European markets commonly appears under titles such as SOC analyst, security analyst, junior detection engineer, security engineer, vulnerability analyst, GRC analyst, application security engineer, cloud security analyst and junior penetration tester. Many "entry-level" listings still ask for foundational IT, networking, scripting, cloud and incident exposure. This roadmap provides project evidence but cannot promise employment, salary, visa sponsorship or relocation. Remote work from Pakistan depends on the employer's legal entity, tax, data-access, time-zone and export-control rules.

Use official frameworks to analyze roles: NIST NICE (https://www.nist.gov/itl/applied-cybersecurity/nice) for task/skill language; NIST CSF 2.0 for control vocabulary; MITRE ATT&CK for detection roles. No live job listing is used in this PDF as proof of demand. If you analyze a real listing, save the URL and retrieval date, then distinguish must-have skills from preferred tools.

Job-description analysis

Make a table with columns: requirement, evidence in your portfolio, gap level (0-3), proof task, interview story. Ignore inflated years-of-experience claims only after you can show equivalent evidence. Classify product names separately from transferable concepts (e.g., "Sentinel" vs "SIEM correlation").

CV and LinkedIn

- One page for an early-career CV; no photo/personal identifiers unnecessary for the target country.
- Top: degree, expected graduation, GitHub/portfolio, location/time-zone, work authorization if relevant.
- Project bullets use problem → action → evidence → result, e.g., "Built a Python parser for 3 synthetic log formats; added 25 pytest cases; reduced malformed-line crashes to zero; documented limitations."
- LinkedIn headline should name target role and evidence, not "ethical hacker" buzzwords.
- Ask for feedback from a teacher, OSS maintainer or working analyst; track changes.

Interview practice

Prepare five stories: one networking/OS troubleshooting, one secure coding fix, one detection/investigation, one cloud/container misconfiguration, one ethical disagreement. For each, state context, hypothesis, evidence, decision, failure, lesson. Practice explaining uncertainty and what data you would request. Do not claim tool mastery from a badge.

Internships and first applications

Apply to internships, university security labs, open-source security projects, CTF teams, local non-profits with written authorization, and junior roles that allow learning. Build an application tracker with date, role, source, requirements, evidence mapping, follow-up date and outcome. Seek feedback, not just offers. For international roles, check official visa/work-authorization sources (e.g., US State Department https://travel.state.gov/content/travel/en/us-visas.html and EU immigration https://home-affairs.ec.europa.eu/), and never assume remote work bypasses work authorization.

Specialization comparison (Month 8-10 decision aid)

Score each row 1-5 for demonstrated interest, project results, programming comfort, local opportunity and available feedback. No path is "best" without your evidence.

Decision framework: evidence weight 35%, genuine interest 20%, programming/technical fit 20%, available mentorship/opportunity 15%, opportunity cost 10%. Require at least two strong project results before selecting. Review after 90 days; switching early because a topic is hard is different from switching because evidence shows poor fit.

Recommendation for this profile: primary = Detection Engineering & Security Automation; secondary = Cloud/Container Security. If Month 8-10 sprints contradict this, choose the path with stronger independent problem-solving evidence and better feedback access.

Project catalogue — 15 required deliverables

Every project includes all required fields. Shared templates T1/T2/T3 and rubric R1 are defined in the preceding text; project rows reference and extend them.

PRJ-01 — Personal cybersecurity learning lab

Problem statement	A beginner needs a safe, reproducible environment but has modest hardware and no paid software license.
Objective	Build a documented, isolated, snapshot-backed lab capable of hosting Linux, Windows, local web apps and browser labs without exposing vulnerable services.
Prerequisites	Virtualization basics, host OS administration, CLI, Git
Architecture	Host with VirtualBox/KVM; Ubuntu VM on NAT plus internal adapter; optional Windows evaluation VM; browser labs; evidence repository
Tools and installation links	VirtualBox https://www.virtualbox.org/manual/; Ubuntu https://help.ubuntu.com/; Git https://git-scm.com/book/en/v2; Wireshark https://www.wireshark.org/docs/wsug_html_chunked/
Step-by-step milestones	1 Inventory hardware; 2 enable virtualization; 3 install hypervisor; 4 create Ubuntu VM; 5 configure NAT/internal adapters; 6 snapshot; 7 optional Windows VM; 8 document setup
Expected results	A clean VM with internet via NAT, an isolated internal adapter, a restore point, and a reproducible README
Validation tests	Boot VM; ping gateway; verify no bridged vulnerable adapter; restore snapshot; compare configuration hash; confirm no host firewall disabled
Common mistakes	Bridging vulnerable VMs; storing personal files in lab; no snapshot; copying random ISO images
Troubleshooting guidance	No boot: enable VT-x/AMD-V; no network: check NAT/DHCP; slow VM: reduce RAM to 2 GB; snapshot corruption: keep exported appliance and backup
Evidence to collect	VM settings export, sanitized screenshots, network diagram, command transcript, snapshot identifier, setup log
Report template	T1 with scope, architecture, setup, validation, limitations, recovery test
GitHub structure	lab-baseline/README.md, docs/, scripts/, evidence/sanitized/
README requirements	Purpose, hardware, exact versions, setup commands, network isolation, restore test, limitations
Security boundaries	Only owned VMs; no public exposure; no personal credentials; host security stays enabled
Grading rubric	R1 standard rubric; special weight on isolation and recovery evidence
Extension challenges	Add Ansible/Vagrant provisioning; add segmented Windows VM; add offline package mirror

PRJ-02 — Security control baseline and risk register

Problem statement	The lab has assets and configurations but no systematic risk-based control selection.
Objective	Apply NIST CSF and CIS-style controls to the student's own lab and measure whether each control works.
Prerequisites	M1-M4 foundations, basic risk vocabulary, VM administration
Architecture	Inventory of VMs, GitHub repo, local web service, synthetic data; controls mapped to Govern/Identify/Protect/Detect/Respond/Recover
Tools and installation links	NIST CSF https://www.nist.gov/cyberframework; CIS Controls https://www.cisecurity.org/controls/v8; NIST SP 800-30 https://csrc.nist.gov/pubs/sp/800/30/r1/final

Step-by-step milestones	1 Asset inventory; 2 data classification; 3 five threats; 4 risk register; 5 select ten controls; 6 implement low-risk controls; 7 test; 8 write exceptions
Expected results	Ten tested controls, a risk register with owners/treatments, and one 90-day improvement plan
Validation tests	Before/after evidence for each control; negative test where control should block; restore test; peer-style review of residual risk
Common mistakes	Confusing policy with working control; no rollback; selecting controls only from a checklist; claiming zero risk
Troubleshooting guidance	If control breaks usability, snapshot and rollback; if evidence is missing, define a measurable test; if risk ranking is inconsistent, state assumptions
Evidence to collect	Asset inventory, risk register, control matrix, test results, screenshots, change log, rollback notes
Report template	T1 plus risk methodology, control-by-control results, exceptions, residual risk
GitHub structure	security-baseline/README.md, docs/threat-model.md, controls/, tests/, evidence/
README requirements	Scope, methodology, how to reproduce each test, limitations, no compliance claim
Security boundaries	Controls applied only to owned lab; no production system; no unauthorized scanning
Grading rubric	R1 with conceptual 25, practical 25, documentation 20, troubleshooting 10, safety 10, independence 10
Extension challenges	Add quantitative risk scoring; automate control tests; compare two frameworks

PRJ-03 — Network diagram and troubleshooting report

Problem statement	A small lab network has connectivity faults and no documented topology or diagnostic method.
Objective	Produce a professional topology and a repeatable OSI-layer troubleshooting report for an isolated network.
Prerequisites	OSI/TCP/IP, CIDR, ARP, routing, Wireshark, Packet Tracer
Architecture	Three-node virtual network with client, server, router/switch simulation; NAT for updates; internal adapter for tests
Tools and installation links	Packet Tracer https://www.netacad.com/courses/packet-tracer; Wireshark https://www.wireshark.org/docs/wsug_html_chunked/; JITL labs https://www.youtube.com/playlist?list=PLxbwE86jKRgMpuZuLBivzlM8s2Dk5lXBQ
Step-by-step milestones	1 Draw physical/logical topology; 2 assign addresses; 3 verify ARP and routes; 4 inject three faults; 5 diagnose layer by layer; 6 fix; 7 retest; 8 report
Expected results	Correct diagram, IP plan, fault timeline, packet evidence and verification tests
Validation tests	Ping/traceroute success; ARP table matches; route table checks; packet capture confirms each fault/fix; diagram matches observed state
Common mistakes	Confusing default gateway and DNS; changing multiple variables at once; ignoring time/ARP cache; no rollback
Troubleshooting guidance	No route: check subnet/default gateway; DNS failure: use IP to isolate; ARP stale: clear cache safely; capture empty: use correct interface
Evidence to collect	Topology diagram, .pkt file or VM config, PCAP excerpts, command logs, before/after test matrix
Report template	T1 plus OSI-layer analysis and evidence table
GitHub structure	network-troubleshooting/README.md, diagrams/, captures/, reports/
README requirements	Exact topology, reproduction commands, sanitized addresses, assumptions, limitations
Security boundaries	Only isolated lab addresses; no campus/public network scanning

Grading rubric	R1 with special weight on troubleshooting and independent hypotheses
Extension challenges	Add IPv6, VLANs, OSPF, or a simulated firewall failure

PRJ-04 — Wireshark traffic-analysis report

Problem statement	Network traffic contains security-relevant evidence, but the student cannot yet distinguish observation from inference.
Objective	Analyze a legal sample PCAP and write a defensible report with confidence levels.
Prerequisites	TCP/IP, DNS, HTTP/TLS, Wireshark display filters
Architecture	Single analysis VM with Wireshark and sample captures; no live user traffic
Tools and installation links	Wireshark Guide https://www.wireshark.org/docs/wsug_html_chunked/; SampleCaptures https://wiki.wireshark.org/SampleCaptures; RFC 9293 https://www.rfc-editor.org/rfc/rfc9293
Step-by-step milestones	1 Hash capture; 2 identify endpoints/conversations; 3 protocol hierarchy; 4 follow streams; 5 identify anomalies; 6 form hypotheses; 7 test with filters; 8 report
Expected results	A report with endpoints, timeline, protocol fields, evidence for each claim and explicit limitations
Validation tests	PCAP hash unchanged; five display filters documented; at least three claims tied to frame numbers; alternative explanation considered
Common mistakes	Treating encrypted payload as plaintext; ignoring time zones; confusing capture and display filters; overclaiming malware
Troubleshooting guidance	No capture: verify interface/permissions; TLS hides fields: use metadata; noisy file: filter by conversation; missing DNS: check external resolver
Evidence to collect	PCAP hash, filter list, annotated screenshots, frame table, timeline, final report
Report template	T1 plus packet-evidence appendix and confidence scale
GitHub structure	traffic-analysis/README.md, captures/, reports/, evidence/
README requirements	Capture source, hash, Wireshark version, filters, findings, limitations
Security boundaries	Use only public sample captures or traffic from owned systems; no third-party interception
Grading rubric	R1 with conceptual 25, practical 25, troubleshooting 15, documentation 15, safety 10, independence 10
Extension challenges	Add Zeek/Suricata comparison; analyze TLS 1.3 metadata; create a synthetic beaconing dataset

PRJ-05 — Linux system-audit script

Problem statement	Manual Linux checks are slow, inconsistent and risky when copied from random blogs.
Objective	Write a read-only Bash/Python audit utility that collects security-relevant system state from an owned Linux VM.
Prerequisites	Linux permissions, processes, services, Bash, Python, JSON
Architecture	Audit script runs with least privilege; outputs common JSON schema; no network calls and no destructive commands
Tools and installation links	BashGuide https://mywiki.wooledge.org/BashGuide; man7 https://man7.org/linux/man-pages/; Python docs https://docs.python.org/3/; CIS Benchmarks https://www.cisecurity.org/cis-benchmarks

Step-by-step milestones	1 Define schema; 2 collect users/groups; 3 permissions; 4 services/listening ports; 5 patch state; 6 logs; 7 JSON output; 8 tests; 9 review
Expected results	A deterministic, read-only tool with tests, sample output and a design document
Validation tests	Run on clean VM; compare output against manual commands; test empty fields, filenames with spaces, missing permissions; shellcheck; no modifications
Common mistakes	Running as root unnecessarily; parsing human-readable text; unquoted variables; exposing /etc/shadow; destructive commands
Troubleshooting guidance	Permission denied: handle gracefully; `ss` missing: use fallback or document; JSON invalid: validate; locale changes: normalize output
Evidence to collect	Script, schema, tests, sample output, manual verification table, shellcheck report, README
Report template	T1 plus control-to-command mapping and limitations
GitHub structure	linux-audit/README.md, src/, tests/, data/synthetic/, docs/
README requirements	Scope, read-only guarantee, setup, usage, schema, limitations
Security boundaries	Run only on owned VM; never publish real user hashes, hostnames or sensitive logs
Grading rubric	R1 with secure coding and least-privilege checks
Extension challenges	Add PowerShell counterpart; add config drift test; add CI and signed release

PRJ-06 — Windows event-log investigation

Problem statement	A synthetic Windows incident needs a defensible timeline, but event IDs are easy to misinterpret.
Objective	Investigate benign simulated authentication/process/service events and write an incident report with confidence levels.
Prerequisites	Windows fundamentals, Event Viewer, PowerShell, Sysmon, evidence integrity
Architecture	Disposable Windows VM; Security/System/Sysmon logs; benign event generator; no real personal data
Tools and installation links	Windows Event Log https://learn.microsoft.com/en-us/windows/win32/wes/windows-event-log; Sysmon https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon; Sysinternals https://learn.microsoft.com/en-us/sysinternals/
Step-by-step milestones	1 Clean snapshot; 2 configure audit/Sysmon; 3 generate logon/service/process events; 4 export EVTX; 5 hash; 6 query fields; 7 timeline; 8 report
Expected results	A reproducible timeline of at least five events with EventID, provider, time, user, process and interpretation
Validation tests	Hash manifest matches; XPath/PowerShell queries reproduce timeline; at least one alternative explanation rejected by evidence
Common mistakes	Ignoring logon type; mixing local and UTC time; relying on EventID alone; editing original EVTX
Troubleshooting guidance	No events: check audit policy; Sysmon absent: verify service; time skew: configure NTP; access denied: use admin shell in lab only
Evidence to collect	Sanitized EVTX, hash manifest, query file, timeline, screenshots, incident report
Report template	T2 incident form with evidence references and confidence labels
GitHub structure	windows-investigation/README.md, events/, queries/, reports/, evidence/
README requirements	Lab scope, event-generation steps, sanitization, tool versions, query instructions
Security boundaries	Disposable VM only; no personal accounts; no live malware; preserve original evidence

Grading rubric	R1 with evidence integrity and false-positive analysis
Extension challenges	Add Sysmon config comparison; add KQL/Sigma rule; add memory acquisition concepts in Year 2

PRJ-07 — Python security-log parser and defensive automation tool

Problem statement	Security logs arrive in multiple formats and contain malformed lines; manual grep is not enough.
Objective	Build a tested parser and analyzer that normalizes synthetic logs and produces defensible summaries.
Prerequisites	Python, regex, files, exceptions, JSON/CSV, SQL, testing, Git
Architecture	CLI tool with parser/analyzer modules; SQLite storage; synthetic inputs; common JSON schema; no external network
Tools and installation links	Python Tutorial https://docs.python.org/3/tutorial/; pytest https://docs.pytest.org/en/stable/; RegexOne https://regexone.com/; SQLite https://www.sqlite.org/docs.html
Step-by-step milestones	1 Define schema; 2 stream input; 3 parse three formats; 4 normalize; 5 store; 6 query top talkers; 7 structured errors; 8 tests; 9 docs
Expected results	Working CLI, normalized dataset, SQL queries, Markdown report and full test suite
Validation tests	20+ tests; 100 MB synthetic file under documented memory cap; malformed lines rejected with reasons; secrets redacted; no shell interpolation
Common mistakes	Catastrophic regex; reading whole file into memory; logging secrets; string-built SQL; no time zone handling
Troubleshooting guidance	Timeout: bound regex and line length; Unicode errors: explicit encoding; DB lock: transaction handling; query slow: index fields
Evidence to collect	Code, schema, sample logs, test results, performance note, redacted output, changelog
Report template	T1 plus data schema, query logic, false-positive discussion, performance
GitHub structure	security-log-parser/README.md, src/, tests/, data/synthetic/, reports/
README requirements	Problem, formats, exact CLI, schema, tests, redaction, limitations
Security boundaries	Synthetic logs only; no production logs; no credential collection
Grading rubric	R1 with secure coding and test quality
Extension challenges	Add property-based tests; add Sigma export; add notebook visualization; add streaming benchmark

PRJ-08 — Secure web application / API

Problem statement	A small notes API must support multiple users without exposing one user's data to another.
Objective	Implement a secure local API with authentication, object ownership, validation, parameterized queries, CSRF protection and structured logging.
Prerequisites	HTTP, SQL, Python, threat modelling, OWASP ASVS, testing
Architecture	Local web app, SQLite/PostgreSQL-like database, session cookies, server-side authz, test suite, CI
Tools and installation links	OWASP ASVS https://owasp.org/www-project-application-security-verification-standard/; Cheat Sheets https://cheatsheetseries.owasp.org/; Python docs https://docs.python.org/3/; SQLite https://www.sqlite.org/docs.html

Step-by-step milestones	1 Requirements; 2 threat model; 3 schema; 4 registration/login/logout; 5 object ownership; 6 anti-CSRF; 7 rate limits; 8 errors/logs; 9 tests; 10 CI
Expected results	Working API with secure defaults, 20+ tests, threat model and reproducible setup
Validation tests	Cross-user access denied; SQL injection strings do not change queries; XSS payload escaped; expired session rejected; CSRF token mismatch rejected; rate limit triggers
Common mistakes	Client-side authz only; plaintext passwords; string SQL; hiding IDs as security; verbose stack traces; secrets in repo
Troubleshooting guidance	Login loop: cookie attributes/domain; 403 unexpectedly: check object owner; CSRF failing: token/session binding; DB locked: transaction scope
Evidence to collect	Source, migrations, threat model, test report, CI log, sanitized request/response examples
Report template	T1 plus security requirements and verification matrix
GitHub structure	secure-api/README.md, src/, migrations/, tests/, docs/, .github/workflows/
README requirements	Scope, setup, roles, endpoints, test commands, threat model, limitations
Security boundaries	Loopback/host-only only; no public deployment; synthetic users only
Grading rubric	R1 with special weight on server-side authorization and negative tests
Extension challenges	Add JWT/OIDC carefully; add CSP/Trusted Types; add SAST/DAST and SBOM

PRJ-09 — Local web application security assessment with remediation

Problem statement	A secure app still needs independent validation and defensive recommendations, not just passing tests.
Objective	Perform an authorized assessment of PRJ-08 on loopback, validate findings, fix root causes and retest.
Prerequisites	PRJ-08, HTTP, OWASP WSTG, PortSwigger basics, secure coding
Architecture	Assessor VM/container, local app, Burp/ZAP proxy, written scope, evidence repository
Tools and installation links	OWASP WSTG https://owasp.org/www-project-web-security-testing-guide/; Burp docs https://portswigger.net/burp/documentation; ZAP docs https://www.zaproxy.org/docs/; Cheat Sheets https://cheatsheetseries.owasp.org/
Step-by-step milestones	1 Scope/RoE; 2 asset inventory; 3 five test areas; 4 validate findings; 5 risk rank; 6 fix; 7 retest; 8 report
Expected results	Three evidence-backed findings, one rejected false positive, before/after tests and a closure memo
Validation tests	Scope contains only 127.0.0.1/internal host; original/variant/regression retests; raw evidence hashes; no destructive action
Common mistakes	Testing without written scope; WAF-only fix; over-ranking scanner alerts; leaving sessions/cookies in report
Troubleshooting guidance	Proxy not intercepting: check browser proxy/cert; app down: verify container; finding not reproducing: compare environment; regression after fix: isolate commit
Evidence to collect	RoE, request/response evidence, scan report, remediation diff, retest results
Report template	T2 assessment report with executive summary, findings, evidence, remediation, retest
GitHub structure	web-assessment/README.md, scope/, evidence/sanitized/, reports/, fixes/
README requirements	Explicit ownership/authorization, sanitized evidence, reproduction steps, limitations
Security boundaries	Only own local app; no third-party target; no public exploit material

Grading rubric	R1 with safety/ethics double weight
Extension challenges	Add authenticated scanning; add business-logic abuse cases; add CVSS/KEV contextual ranking

PRJ-10 — Authorized vulnerability assessment report for an isolated lab

Problem statement	A realistic assessment needs planning, discovery, validation, communication and closure under a written scope.
Objective	Execute an end-to-end assessment of two owned VMs and one local vulnerable web app, then report and retest.
Prerequisites	M1-M7 prerequisites, networking, Linux/Windows, web security, ethics
Architecture	Two local VMs, one vulnerable app, NAT/internal network, Nmap/Burp/Wireshark, evidence repo
Tools and installation links	NIST SP 800-115 https://csrc.nist.gov/pubs/sp/800/115/final; Nmap Reference https://nmap.org/book/man.html; WSTG https://owasp.org/www-project-web-security-testing-guide/; FIRST CVSS https://www.first.org/cvss/
Step-by-step milestones	1 Charter; 2 RoE; 3 inventory; 4 threat model; 5 discovery; 6 validation; 7 evidence; 8 report; 9 remediation; 10 retest
Expected results	A professional report with prioritized findings, false-positive analysis, remediation tickets and closure evidence
Validation tests	All targets in scope; three findings with raw evidence; CVSS plus business context; no persistence/lateral movement beyond minimal proof; retest closes root cause
Common mistakes	Scope creep; destructive proof; scanner dependence; missing evidence hash; confusing version with vulnerability
Troubleshooting guidance	Host down: stop and verify; scanner crash: reduce rate; ambiguous banner: second validation; report disagreement: cite raw evidence
Evidence to collect	Scope, asset inventory, scan outputs, PCAPs, screenshots, evidence manifest, final report
Report template	T2 full assessment template with appendices
GitHub structure	vuln-assessment/README.md, scope/, evidence/sanitized/, reports/, remediation/
README requirements	Scope statement, sanitized artifacts, tool versions, limitations
Security boundaries	Isolated owned systems only; no public IPs; no real user data
Grading rubric	R1 with safety/ethics threshold 10/10
Extension challenges	Add credentialed scanning; add AD lab; add CVSS environmental scoring; add retest automation

PRJ-11 — SIEM alert investigation and incident report

Problem statement	A synthetic alert must be triaged, investigated and communicated without overclaiming from limited logs.
Objective	Build a reproducible incident investigation from synthetic Windows/Linux/cloud logs and write a professional report.
Prerequisites	Log normalization, KQL/SQL, ATT&CK, IR lifecycle, evidence integrity
Architecture	Local dataset or free sample workspace; normalized event table; KQL-style queries; timeline; case management notes

Tools and installation links	KQL docs https://learn.microsoft.com/en-us/kusto/query/; Sentinel docs https://learn.microsoft.com/en-us/azure/sentinel/; NIST 800-61r3 https://csrc.nist.gov/pubs/sp/800/61/r3/final; ATT&CK https://attack.mitre.org/
Step-by-step milestones	1 Define alert; 2 generate/download data; 3 normalize; 4 triage; 5 eight queries; 6 timeline; 7 ATT&CK map; 8 response; 9 report
Expected results	Incident report with timeline, confidence levels, root-cause hypothesis, evidence gaps and response plan
Validation tests	All timestamps normalized; eight queries reproducible; one benign alternative rejected; evidence hashes; no cloud overspend
Common mistakes	Assuming an IOC is an incident; mixing time zones; query joins without cardinality check; ignoring data gaps
Troubleshooting guidance	No tables: use local SQLite equivalent; high false positives: narrow entity/time; missing host: state limitation; cost alert: stop ingestion
Evidence to collect	Data manifest, queries, timeline, screenshots, incident report, executive summary, detection-gap notes
Report template	T2 incident report with known/likely/unknown sections
GitHub structure	siem-investigation/README.md, data/synthetic/, queries/, reports/, evidence/
README requirements	Dataset provenance, time zone, schema, query setup, sanitization
Security boundaries	Synthetic data only; if cloud used, budgets and teardown documented
Grading rubric	R1 with uncertainty/documentation weight
Extension challenges	Add SOAR-style playbook; add KQL to Sigma conversion; add tabletop exercise

PRJ-12 — Detection rule with test data

Problem statement	A detection idea is not credible until it is rule-formatted, tested on true/false positives and maintained as code.
Objective	Write an original Sigma rule for a lab behaviour, convert it to a query and validate on synthetic events.
Prerequisites	ATT&CK, logs, KQL/SQL, Sigma, Git, testing
Architecture	Detection-as-code repo; synthetic event generator; rule; conversion script; tests; CI-like local validation
Tools and installation links	SigmaHQ https://sigmahq.io/; Sigma guide https://sigmahq.io/docs/guide/getting-started.html; Elastic detections https://www.elastic.co/guide/en/security/current/detection-engine-overview.html; ATT&CK https://attack.mitre.org/
Step-by-step milestones	1 Choose technique; 2 data requirements; 3 rule; 4 ten positives/ten negatives; 5 convert; 6 backtest; 7 tune; 8 document bypasses
Expected results	A rule, labelled dataset, conversion output, metrics, false-positive notes and version history
Validation tests	Rule validates; all assigned true positives detected; false positives below documented threshold; at least three bypasses listed; CI test fails on broken rule
Common mistakes	Blanket suppression; untested field names; selecting only easy positives; no rule owner; confusing precision with value
Troubleshooting guidance	Conversion error: check backend/pipeline; no matches: verify field normalization; too many alerts: prevalence analysis; rule valid but useless: revisit hypothesis
Evidence to collect	Rule YAML, test events, conversion output, metrics, changelog, design document, ATT&CK mapping
Report template	T1 plus detection design document with hypothesis, data, tests, bypasses

GitHub structure	detection-rule/README.md, rules/, tests/data/, docs/, scripts/, evidence/
README requirements	Technique, logsource, setup, test commands, metrics, limitations
Security boundaries	Synthetic/authorized telemetry only; no deployment on production systems
Grading rubric	R1 with special weight on false-positive evidence
Extension challenges	Add correlation rule; add multi-backend conversion; add ATT&CK Navigator layer; add 10,000-event backtest

PRJ-13 — Cloud configuration security review

Problem statement	A fictional small cloud environment has IAM/network/storage/logging weaknesses but no budget for live deployment.
Objective	Review synthetic architecture and configuration artifacts, prioritize risks and provide verification steps.
Prerequisites	Cloud shared responsibility, IAM, storage, logging, risk, documentation
Architecture	Fictional AWS/Azure account structure with ten resources; JSON policy samples; architecture diagram; no billable resources
Tools and installation links	AWS Security Pillar https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html; AWS security docs https://docs.aws.amazon.com/security/; Azure security https://learn.microsoft.com/en-us/azure/security/; CSA CCM https://cloudsecurityalliance.org/research/cloud-controls-matrix
Step-by-step milestones	1 Inventory; 2 classify data; 3 IAM review; 4 network review; 5 storage; 6 logging; 7 risk register; 8 remediation; 9 verification
Expected results	Twelve findings with blast-radius analysis, owner, verification command/query and residual risk
Validation tests	All findings supported by synthetic config lines; policy logic checked; no live cloud resource created; risk priorities include business context
Common mistakes	Assuming encryption fixes public access; treating NAT as firewall; ignoring control-plane logs; copy-pasting another cloud's defaults
Troubleshooting guidance	Policy ambiguous: draw trust graph; missing logs: state visibility gap; service names changed: verify current docs; disputed risk: define scenario
Evidence to collect	Architecture diagram, synthetic policies, risk register, review checklist, remediation plan, references
Report template	T2 assessment report plus control-mapping appendix
GitHub structure	cloud-review/README.md, architecture/, configs/synthetic/, reports/, checks/
README requirements	Explicit fictional scope, no live account, assumptions, verification steps
Security boundaries	No real customer data; no cloud credentials; no public deployment
Grading rubric	R1 with conceptual/architecture weight
Extension challenges	Add Checkov/Terraform scan; add AWS/Azure comparison; add detection hypothesis for each high risk

PRJ-14 — Docker image and container security review

Problem statement	A container image may contain vulnerable packages, secrets, excessive privileges and insecure networking.

Objective	Build/review a local image and runtime configuration, fix findings and rescan with reproducible evidence.
Prerequisites	Docker basics, Linux permissions, CVE/SBOM, CI, scripting
Architecture	Local Docker/Podman host; intentionally weak image; Trivy scanner; runtime metadata; isolated bridge network
Tools and installation links	Docker docs https://docs.docker.com/; Docker security https://docs.docker.com/engine/security/; Trivy https://trivy.dev/docs/; OWASP K8s Top 10 https://owasp.org/www-project-kubernetes-top-ten/
Step-by-step milestones	1 Snapshot baseline; 2 inspect layers; 3 scan; 4 Dockerfile review; 5 runtime review; 6 fix; 7 rescan; 8 report
Expected results	Ten findings, five fixes, before/after scans, runtime tests and residual-risk notes
Validation tests	Hardened image runs non-root; no secret in history; capabilities dropped; read-only root where feasible; network isolation verified
Common mistakes	Using latest tag blindly; privileged container; secrets in ENV/ARG; scanning without context; leaving volumes world-writable
Troubleshooting guidance	Build fails: pin base image; app needs write: use tmpfs; scanner DB blocked: use offline/ documented limitation; container exits: inspect logs
Evidence to collect	Dockerfiles, image IDs/digests, scan JSON, runtime inspect output, test log, diff
Report template	T2 assessment report with image inventory and finding-by-finding evidence
GitHub structure	container-review/README.md, docker/, scans/, tests/, reports/evidence/
README requirements	Local-only scope, image versions, reproduction commands, sanitized outputs
Security boundaries	No public registry push; no host socket mount; no production daemon
Grading rubric	R1 with runtime-isolation and vulnerability-triage weight
Extension challenges	Add SBOM/provenance; add rootless Docker; add Kubernetes admission policy; add CI gate

PRJ-15 — Final integrated capstone

Problem statement	The student needs one coherent project that combines secure engineering, telemetry, detection, incident response, cloud/container controls and independent research.
Objective	Design, build, attack safely, detect, respond, document and defend an original system aligned with the chosen specialization.
Prerequisites	M1-M11, chosen specialization, primary docs, technical writing
Architecture	Local three-tier app/API, structured logging, three detections, hardened container, synthetic cloud/ IaC representation, CI and evidence repo
Tools and installation links	OWASP ASVS https://owasp.org/www-project-application-security-verification-standard/; SigmaHQ https://sigmahq.io/; AWS Security Pillar https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html; GitHub Actions hardening https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions; NIST 800-61r3 https://csrc.nist.gov/pubs/sp/800/61/r3/final
Step-by-step milestones	1 Charter; 2 threat model; 3 architecture; 4 reproducible setup; 5 implementation; 6 detections; 7 failure injection; 8 authorized assessment; 9 remediation; 10 report/demo
Expected results	Working system, labelled benign/malicious data, tested detections, a security assessment, incident report, reproducible setup, demo and limitations
Validation tests	Clean install succeeds; 25+ tests; three detections with true/false-positive data; failure-injection tests; assessment retests; report passes 75+
Common mistakes	

	Oversized scope; unlabelled synthetic data; no negative results; tutorial-replica architecture; overclaiming production readiness
Troubleshooting guidance	Performance: reduce components; missing telemetry: instrument first; false positives: labelled baseline; report too broad: narrow claim; demo fails: use tagged release
Evidence to collect	Source, manifests, data dictionary, test outputs, evidence manifest, threat model, report, demo script, changelog
Report template	T3 capstone report: abstract, problem, related work, design, implementation, evaluation, security assessment, IR, limitations, future work
GitHub structure	capstone/README.md, docs/, src/, detections/, data/synthetic/, tests/, evidence/sanitized/, .github/workflows/
README requirements	One-sentence contribution, architecture, exact setup, demo, tests, limitations, ethics statement
Security boundaries	All targets owned/local; synthetic data; no public vulnerable service; no secrets; no unauthorized exploit material
Grading rubric	R1 plus capstone defence: 20 concept, 25 practical, 15 independence, 15 troubleshooting, 15 documentation, 10 safety; pass 75
Extension challenges	Publish a dataset; add multi-cloud comparison; add a research reproduction; submit a conference-style extended abstract

Printable trackers

Daily checklist (copy one per day)

#	Item	Done	Evidence / note
1	Read today's card and objective	[ ]	
2	Primary resource section completed	[ ]	
3	Three facts / two questions / one command	[ ]	
4	Practical task completed	[ ]	
5	One error observed or injected	[ ]	
6	Retrieval answer written without notes	[ ]	
7	Evidence file saved and sanitized	[ ]	
8	Journal / Git commit	[ ]	
9	Track used: Core / Standard / Intensive	[ ]	
10	Weak cards scheduled 1/3/7/30/90 days	[ ]	

Weekly tracker

Monthly scorecard (100 points)

Error log template

Command notebook template

Command / API	Platform	Purpose	Safe example	Dangerous assumption	Verification	Source URL

Project log template

Date	Decision	Alternatives	Evidence	Risk	Result	Follow-up

Final completion checklist

- [ ] 365 daily cards have evidence and retrieval notes.
- [ ] 12 monthly scorecards complete; M3/M6/M9/M12 gates passed.
- [ ] 15 projects have a problem, architecture, tests, evidence, report, README, boundaries and limitations.
- [ ] Final capstone has a clean-install test, threat model, detections, incident report, assessment, remediation and demo.
- [ ] All public artifacts are sanitized; no secrets, private logs, personal data or unauthorized exploit material.
- [ ] Resource links rechecked on 2026-10-27; backup links documented.
- [ ] Certification decision uses current official pages and a written reason.
- [ ] CV, LinkedIn and interview stories map to portfolio evidence.
- [ ] Year 2 plan has a primary specialization, secondary skill, peer group, 90-day sprint and review date.
- [ ] Lab is clean, isolated, backed up and has no active billable cloud resources.

Landscape reference table

Path	Foundational knowledge	Daily work	Technical depth / programming	Common tools	Strong free resources	Portfolio expectation	Possible entry roles	Long-term opportunities	Typical prerequisites	12-month limitation
Blue Team / SOC Analysis	Networking, OS, logs, ATT&CK	Alert triage, investigation, escalation, reporting	Medium; scripting helpful	SIEM, KQL, EDR, ticketing	Microsoft Learn, SigmaHQ, CyberDefenders, BTLO	Alert investigation, incident report, detection tests	SOC analyst, security analyst	Detection lead, IR, threat hunting	M1-M8 basics	No real shift experience or production scale
Detection Engineering	Logs, OS internals, ATT&CK, SQL/KQL	Hypothesis, rule design, testing, tuning, deployment	High; Python/YAML/SQL	Sigma, KQL, Elastic rules, Git, CI	SigmaHQ, KQL docs, Elastic rules, ATT&CK	Rule repo with labelled true/false positives and metrics	Junior detection engineer, SOC analyst	Detection platform/ architecture, research	Strong logs and programming	Lab detections may not generalize to enterprise data
DFIR	Filesystems, Windows/Linux artifacts, evidence integrity	Acquisition, timeline, artifact analysis, reporting	High; scripting and OS internals	Autopsy, KAPE-like tools, Volatility, Wireshark	Autopsy, SANS webcasts, NIST 800-61r3, EVTX samples	Disk/network/ memory case with hash manifest	Junior DFIR analyst, IR analyst	Senior IR, forensic research, expert witness	Evidence handling and OS internals	No legal/casework or real incident pressure
Web / API Security	HTTP, auth, databases, secure coding	Code review, testing, remediation, threat modelling	High; one backend language	Burp/ZAP, Semgrep, OWASP ASVS, CI	PortSwigger, OWASP, MDN, GitHub security	Secure app plus authorized assessment/ remediation	AppSec analyst, secure software engineer	AppSec architecture, research, product security	M1-M6 plus testing	Advanced vulnerability classes need years
Authorized Penetration Testing	Networking, OS, web, scripting, ethics	Scoped discovery, validation, reporting, retest	Medium-high; scripting and OS	Nmap, Burp, AD tools, reporting templates	NIST 800-115, HTB free, pwn.college, local AD lab	Professional scoped report and remediation retest	Junior pentester, security consultant	Senior pentest, red team, security research	Strong M1-M7 + report writing	Certification or CTF badges do not equal client judgement
Cloud Security	IAM, networking, logging, containers, shared responsibility	Architecture review, policy, detection, incident response	High; IaC and automation	AWS/Azure docs, Trivy, Checkov, K8s	AWS Security Pillar, Azure security, K8s docs	Synthetic cloud review + IaC/CI hardening	Cloud security analyst, DevSecOps junior	Cloud security architect, platform engineering	M1-M9 and Linux/ Docker	No production multi-account experience in Year 1
Security Engineering	Secure design, coding, identity, systems, automation	Build controls, review architecture, write tools	Very high; software engineering	Git, CI, SAST/SCA, cloud, observability	OWASP ASVS, GitHub docs, NIST, Security Engineering	Maintained tool/ library with tests and design doc	Security engineer, platform security engineer	Staff/principal security architect	Strong CS and programming	Breadth and production trade-offs take years
GRC	Risk, policy, controls, audits, privacy	Assessments, evidence, exceptions, reports	Low-medium technically; analysis-heavy	GRC platforms, spreadsheets, NIST/ISO	NIST CSF/SP, CIS Controls, GDPR text	Risk register, control baseline, audit-style report	GRC analyst, risk analyst, compliance associate	Security governance lead, privacy, audit	Business/legal context and strong writing	Framework knowledge without operations is shallow
Vulnerability Research	C/assembly, OS, memory, protocols, fuzzing	Root-cause analysis, PoC in authorized lab, disclosure	Very high; C/ Python/assembly	Ghidra, GDB, pwntools, fuzzers, VMs	pwn.college, OST2, MIT 6.858, Project Zero	Reproduced bug class, detailed root-cause write-up	Rare junior roles; often via internships/OSS	Researcher, product security, exploit engineering	Several years of low-level foundations	Intro exposure is not research capability
Malware Analysis / RE	x86/x64, Windows internals, networking, scripting	Triage, static/ dynamic analysis, indicator extraction	Very high; C/ Python/assembly	Ghidra, x64dbg, Wireshark, sandboxes	OST2, Malware Unicorn RE101, pwn.college	Safe synthetic sample analysis with isolation evidence	Junior malware analyst, SOC/IR specialist	Threat research, reverse engineering	C/assembly and strict lab safety	Real malware handling requires supervised environment

Landscape reference table

Day	Sequence	Hours	Evidence link	Concept /5	Practical /5	Troubleshooting /5	Safety pass	Error log ID	Next action
1							[ ]		
2							[ ]		
3							[ ]		
4							[ ]		
5							[ ]		
6							[ ]		
7							[ ]		

Landscape reference table

Category	Max	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12
Conceptual understanding	20												
Practical execution	25												
Independent problem-solving	15												
Troubleshooting	15												
Documentation	15												
Safety and ethics	10												
Total	**100**												
Gate result				**M3**			**M6**			**M9**			**M12**

Landscape reference table

ID	Date	Symptom	Hypothesis	Test	Result	Root cause	Fix	New test	Retest